

1 Explainability in Reinforcement Learning

2 Maxime Alaarabiou

3 Thursday 10th September, 2026

Chapter 1

Experiments

This chapter reports the experimental study of the Clustering Agent Representation (CAR) method introduced in Chapter ???. While the previous chapter presented CAR and its evaluation metrics from a theoretical standpoint, this chapter focuses on their concrete realization: the software used to run the experiments, the environments and training setups considered, and a detailed analysis of the results obtained.

This chapter is organized as follows:

- Section 1.1 describes the software infrastructure implementing the CAR pipeline.
- Section 1.2 details the experimental configuration, including the environments, the reinforcement learning setup, and the training of the surrogate policies.
- Section ?? presents and analyses the results obtained by applying CAR and its metrics.
- Section ?? summarizes the findings and discusses their implications.

1.1 Software Infrastructure

Deep Learning (DL) and Reinforcement Learning (RL) are inherently empirical fields, in which the effectiveness of a method is ultimately assessed through experimentation. Experimental validation is therefore as important as the theoretical formulation of the method. For this reason, particular attention is given to the implementation of CAR. The method relies on established libraries that provide efficient and widely used implementations of its main components. The implementation is made publicly available as open-source software to support the objective and reproducible replication of the reported results.¹

CAR is organized as a pipeline of five stages, shown in Figure 1.1. Each stage is an independent executable, and stages communicate only through artifacts written to disk. This decoupling allows any stage to be rerun or inspected independently. The parameters of each experiment are gathered in a

¹https://github.com/Maxalaar/Clustering_Agent_Latent_Space

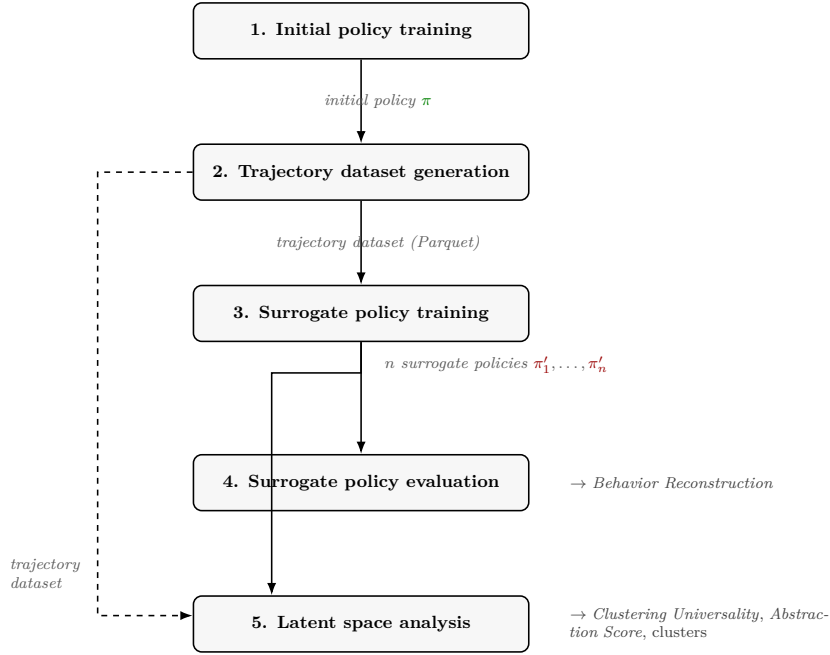


Figure 1.1: The CAR pipeline. Each stage is an independent executable; stages communicate only through artifacts written to disk (solid arrows). The dashed arrow indicates an artifact reused by a later stage.

single configuration object, described in Section 1.2. Each stage also saves the configuration it used alongside its outputs. Every result can therefore be traced back to the settings that produced it.

1.1.1 Initial policy training

The first stage trains the initial policy π using RLlib [Moritz et al., 2018, Liang et al., 2018]. RLlib provides a highly scalable implementation of RL algorithms and supports a wide range of state-of-the-art methods. It supports distributed training, allowing the computational workload to be spread across multiple processes and machines.

The use of RLlib allows CAR to be applied to any environment implemented using the Gymnasium interface [Towers et al., 2024]. This provides a common interface for environments with different observation and action spaces. This flexibility allows CAR to be applied to a wide range of environments.

By default, the RL algorithm used for the initial policy is Proximal Policy Optimization (PPO) [Schulman et al., 2017]. Other algorithms supported by RLlib, such as Asynchronous Proximal Policy Optimization (APPO) [Espeholt et al., 2018], Deep Q-Networks (DQN) [Mnih et al., 2013], and DreamerV3, can also be selected through the same configuration.

The policy and critic networks are implemented using PyTorch [Paszke et al., 2019]. PyTorch provides efficient GPU computation and supports a wide range of neural network architectures. This provides a standardized implementation that is compatible with the broader DL ecosystem.

The output of this stage is a checkpoint containing the trained policy π .

1.1.2 Trajectory dataset generation

The second stage builds the dataset used to train the surrogate policies. The distributed execution capabilities of RLlib are used to collect trajectories in parallel. Multiple environment runners execute the original policy simultaneously, distributing the computational workload across the available resources.

The original policy is run in the environment, and the interactions between the agent and the environment are recorded. For each step, the dataset stores the observation together with the parameters of the action distribution produced by the policy. Trajectories are stored as Parquet files [The Apache Software Foundation, 2016]. Parquet is a compressed columnar format widely used for large-scale data processing. Its columnar structure allows data to be read efficiently and supports concurrent access from multiple processes. This is particularly useful here because the datasets contain on the order of one million observation–action pairs and cannot be loaded entirely into memory.

1.1.3 Surrogate policy training

The third stage trains the surrogate policies using behavior cloning. This stage therefore converts the original RL problem into a supervised learning problem. The surrogate policies are trained to reproduce the action distributions generated by π from the collected observations.

The training procedure is implemented using PyTorch Lightning [Falcon and The PyTorch Lightning team, 2019]. PyTorch Lightning provides a structured interface for supervised training while remaining compatible with the PyTorch ecosystem. It handles common training operations such as data loading, optimization, checkpointing, and GPU execution, while allowing the underlying neural network implementation to remain fully controlled.

Once training is complete, the stage produces a set of n trained surrogate policies.

1.1.4 Surrogate policy evaluation

The fourth stage measures how faithfully each surrogate policy reproduces the behavior of π . Each surrogate is wrapped so that it can be used as an RLlib policy and is then evaluated in the original environment. Its mean cumulative reward over multiple episodes is compared with that of π . Evaluation episodes are collected using parallel environment runners.

This stage provides the value of the *Behavior Reconstruction*. It therefore verifies whether the surrogate policies adequately reproduce the behavior of the original policy before their latent representations are analyzed. If a surrogate does not reproduce the original behavior sufficiently well, its training can be repeated before proceeding to the next stage.

1.1.5 Latent space analysis

The fifth stage performs the main analysis of CAR. The latent representations produced by the surrogate policies form the point clouds that are analyzed using clustering algorithms.

The clustering step relies on the cuML library [RAPIDS Development Team, 2023], which provides GPU-accelerated implementations of several clustering algorithms.

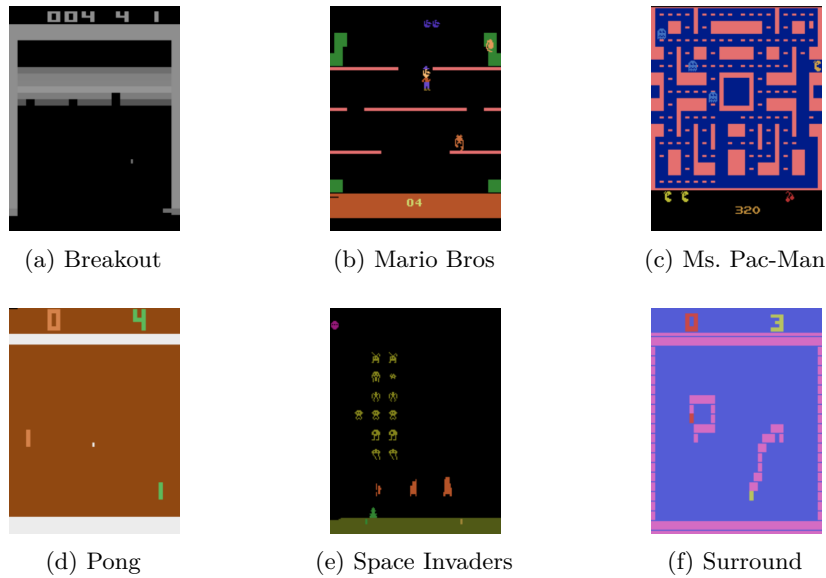


Figure 1.2: The six Atari environments used to evaluate CAR, shown as rendered game frames. The agent does not receive these frames directly: each observation is a stack of four consecutive frames, resized to 64×64 pixels and converted to grayscale, except for Ms. Pac-Man, which is kept in color.

This significantly reduces the computational cost of clustering and makes it possible to analyze point clouds containing tens of thousands of representations.

The resulting partitions are used to compute the *Clustering Universality* and the *Abstraction Score*. The outputs of this stage are the resulting metric values and the material required for the qualitative analysis presented in the following sections.

1.2 Experimental Configuration

This section describes the setup shared by all the experiments reported in this chapter: the environments, the neural architecture of the policies, and the main parameters of each pipeline stage. For every environment, a single initial policy is trained and then imitated by $n = 3$ surrogate policies. The initial policies are not averaged over several reinforcement learning seeds, a limitation discussed in Section ?? . Only the settings common to all experiments are reported here; the exact per-environment values are available in the released configuration files.

1.2.1 Environments

All experiments use environments from the Atari benchmark [Bellemare et al., 2013], accessed through the Arcade Learning Environment and configured following common practice in reinforcement learning research [Machado et al., 2017]. In every environment the agent receives visual observations and selects actions from a small discrete set. The six environments are shown in Figure 1.2.

120 **Breakout.** The agent moves a paddle at the bottom of the screen (Figure 1.2a)
 121 to bounce a ball into a wall of bricks, each broken brick scoring a point, and the
 122 ball speeding up as the wall is cleared. The observation is dominated by the
 123 layout of the remaining bricks, which changes gradually over a game.

124 **Mario Bros.** A single-screen platform game (Figure 1.2b). The player clears
 125 each phase by hitting enemies from below to flip them, then kicking them off
 126 the platforms before they recover. A central POW block flips every enemy at
 127 once but can only be used a few times per phase. Deciding when to spend the
 128 POW block is a clear strategic choice, which makes this environment useful for
 129 testing whether CAR recovers strategy-level distinctions.

130 **Ms. Pac-Man.** The player moves through a maze (Figure 1.2c), eating pellets
 131 while avoiding four ghosts, with power pellets briefly making the ghosts edible.
 132 The maze structure and the many possible local situations, such as a nearby
 133 ghost, an intersection, or an active power pellet, produce a large number of
 134 distinguishable states. This makes Ms. Pac-Man well suited to examining CAR
 135 at fine cluster granularities. It is also the only environment kept in color.

136 **Pong.** A minimal two-player game (Figure 1.2d) in which the agent controls
 137 one paddle and must send the ball past the opponent. The observation contains
 138 almost nothing beyond the two paddles, the ball, and the score. Pong therefore
 139 acts as a lower bound on how much structure CAR can be expected to recover.

140 **Space Invaders.** The agent moves a cannon along the bottom of the screen
 141 (Figure 1.2e), firing at descending rows of aliens while dodging their shots and
 142 using destructible shields, with a bonus saucer occasionally crossing the top. A
 143 game goes through visually distinct phases, from the full alien formation to a
 144 few fast aliens near the ground, which makes this environment convenient for
 145 illustrating the clusters found by CAR.

146 **Surround.** A two-player game on a grid (Figure 1.2f). Each player leaves a
 147 solid trail behind a moving cursor, and the goal is to force the opponent to crash
 148 into a trail or a wall. Every move permanently blocks a cell, so the state of a
 149 game is essentially the pattern of trails drawn so far, which grows continuously
 150 over an episode.

151 1.2.2 Neural architecture

152 The initial policy and the surrogate policies are convolutional networks that share
 153 the same feature-extraction trunk. The trunk is a sequence of four convolutional
 154 layers, with 16, 32, 64, and 128 channels, 4×4 kernels, and a stride of 2, each
 155 followed by layer normalization and a LeakyReLU activation. Its input is the
 156 observation, a stack of four consecutive 64×64 frames.

157 The initial policy follows an actor-critic design: the trunk feeds two separate
 158 multilayer perceptrons, one producing the parameters of the action distribution
 159 and one producing the value estimate. The surrogate policy has the encoder-
 160 decoder structure introduced in Section ??: its encoder f_{enc} is a trunk with the
 161 same architecture, and its decoder is a single multilayer perceptron producing

Observations	
Frame size	64×64
Frames stacked per observation	4
Frame skip	5
Sticky-action probability	0.05
Color	grayscale (color for Ms. Pac-Man)
Initial policy training (PPO)	
Optimizer	Adam
Learning rate	1.5×10^{-4}
GAE λ	0.95
Policy clipping ε	0.1
Value-function clipping	10
Entropy coefficient	0.01
KL coefficient	0.5
Epochs per update	10
Gradient clipping (global norm)	100
Surrogate policy training (behavior cloning)	
Surrogate policies per environment	3
Optimizer	Adam
Learning rate	1×10^{-4}
Batch size	2048
Loss	MSE between action-distribution parameters
Train / validation split	80 / 20
Latent space analysis	
Observations per surrogate policy	50 000
Clustering algorithm	k -means, Euclidean distance, 10 restarts
Number of clusters K	2 to 30
Points for the 2D projection	10 000

Table 1.1: Parameters common to all experiments, grouped by pipeline stage.

the parameters of the action distribution. The surrogate has no value head, since it is trained only to reproduce the action distribution of the initial policy.

The latent space analyzed by CAR is the representation produced by the last convolutional layer of the surrogate encoder.

1.2.3 Parameters

Table 1.1 lists the parameters common to all experiments, grouped by pipeline stage. The trajectory dataset generated for each initial policy contains on the order of one million observation–action pairs. Parameters that vary across environments, such as the reward scaling, the exact number of training epochs, and the dataset size, are given in the released configuration files.

172 Acronyms

173 **APPO** Asynchronous Proximal Policy Optimization 2

174 **CAR** Clustering Agent Representation 1–6

175 **DL** Deep Learning 1, 2

176 **DQN** Deep Q-Networks 2

177 **PPO** Proximal Policy Optimization 2

178 **RL** Reinforcement Learning 1–3

Bibliography

- [Bellemare et al., 2013] Bellemare, M. G., Naddaf, Y., Veness, J., and Bowling, M. (2013). The arcade learning environment: An evaluation platform for general agents. Journal of Artificial Intelligence Research, 47:253–279. **Article.**
- [Espeholt et al., 2018] Espeholt, L., Soyer, H., Munos, R., Simonyan, K., Mnih, V., Ward, T., Doron, Y., Firoiu, V., Harley, T., Dunning, I., Legg, S., and Kavukcuoglu, K. (2018). IMPALA: Scalable distributed deep-RL with importance weighted actor-learner architectures. In Proceedings of the 35th International Conference on Machine Learning (ICML).
- [Falcon and The PyTorch Lightning team, 2019] Falcon, W. and The PyTorch Lightning team (2019). PyTorch Lightning. <https://github.com/Lightning-AI/pytorch-lightning>.
- [Liang et al., 2018] Liang, E., Liaw, R., Nishihara, R., Moritz, P., Fox, R., Goldberg, K., Gonzalez, J. E., Jordan, M. I., and Stoica, I. (2018). RLlib: Abstractions for distributed reinforcement learning. In Proceedings of the 35th International Conference on Machine Learning (ICML).
- [Machado et al., 2017] Machado, M. C., Bellemare, M. G., Talvitie, E., Veness, J., Hausknecht, M., and Bowling, M. (2017). Revisiting the arcade learning environment: Evaluation protocols and open problems for general agents. **Article. Web.**
- [Mnih et al., 2013] Mnih, V., Kavukcuoglu, K., Silver, D., Graves, A., Antonoglou, I., Wierstra, D., and Riedmiller, M. (2013). Playing atari with deep reinforcement learning.
- [Moritz et al., 2018] Moritz, P., Nishihara, R., Wang, S., Tumanov, A., Liaw, R., Liang, E., Elibol, M., Yang, Z., Paul, W., Jordan, M. I., and Stoica, I. (2018). Ray: A distributed framework for emerging AI applications. In 13th USENIX Symposium on Operating Systems Design and Implementation (OSDI), pages 561–577.
- [Paszke et al., 2019] Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Köpf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., Bai, J., and Chintala, S. (2019). PyTorch: An imperative style, high-performance deep learning library. In Advances in Neural Information Processing Systems (NeurIPS).

- 214 [RAPIDS Development Team, 2023] RAPIDS Development Team (2023).
215 RAPIDS cuML: GPU-accelerated machine learning. [https://github.com/](https://github.com/rapidsai/cuml)
216 [rapidsai/cuml](https://github.com/rapidsai/cuml).
- 217 [Schulman et al., 2017] Schulman, J., Wolski, F., Dhariwal, P., Radford, A., and
218 Klimov, O. (2017). Proximal policy optimization algorithms.
- 219 [The Apache Software Foundation, 2016] The Apache Software Foundation
220 (2016). Apache Arrow and Parquet. <https://arrow.apache.org>.
- 221 [Towers et al., 2024] Towers, M., Kwiatkowski, A., Terry, J., Balis, J. U.,
222 De Cola, G., Deleu, T., Goulão, M., Kallinteris, A., Krimmel, M., KG,
223 A., et al. (2024). Gymnasium: A standard interface for reinforcement learning
224 environments.